VENDING MACHINE

1.AIM

Design and Simulation of a Vending Machine Controller Using Verilog HDL

2. Introduction

Vending machines are automated systems that dispense products in exchange for payment, requiring robust and reliable control logic. This project presents the design and simulation of a vending machine controller using Verilog HDL. The controller supports five products with different prices, accepts coins or online payments, handles cancellations, and returns change. The design demonstrates the use of finite state machines (FSM) in hardware control, modular design, and simulation-based verification.

3. Modules

This vending machine controller consists of a single main module implementing all logic. The core functionalities are:

- **Product Selection:** Allows the user to select from five products.
- **Payment Handling:** Accepts coin values or online payment confirmation.
- **State Machine Controller:** Manages the vending process, including product selection, payment validation, dispensing, and change return.
- **Change Calculation:** Calculates and returns change if excess money is inserted.
- **Cancel Handling:** Returns all inserted money if the transaction is cancelled.

4. Design Sources and Their Elaborated Designs

4.1 Vending Machine Controller (FSM)

```
`timescale 1ns / 1ps

module vending_machine

  #(

    parameter KITKAT_PRICE          =7'd20,

    parameter SNICKERS_PRICE        =7'd30,
```

```
    parameter FIVESTAR_PRICE          =7'd10,
    parameter DIARYMILK_PRICE         =7'd40,
    parameter FUSE_PRICE              =7'd35
    )
   (
    //Global Signals
    input wire          i_clk,                //Clock signal (100MHz)
    input wire          i_rst,                //Reset signal (Active High)

    //Inputs
    input wire          i_start,              //Initialize System
    input wire          i_cancel,             //Cancel signal
    input wire [2:0]      i_product_code,         //Product selection
    input wire [6:0]      i_total_coin_value,     //Total value of the coin
    input wire          i_online_payment,       //Conformation of the online payment

    //Outputs
    output wire [3:0]     o_state,                //Present state
    output wire          o_dispense_product,     //whether product dispensed
    output wire [6:0]     o_product_price,        //product price
    output wire [6:0]     o_return_change         //returned change
   );

//States
localparam IDLE_STATE                 =4'b0000,
          SELECT_PRODUCT_STATE         =4'b0001,
```

```
        KITKAT_SELECTION_STATE        =4'b0010,
        SNICKERS_SELECTION_STATE      =4'b0011,
        FIVESTAR_SELECTION_STATE      =4'b0100,
        DIARYMILK_SELECTION_STATE     =4'b0101,
        FUSE_SELECTION_STATE          =4'b0110,
        DISPENSE_AND_RETURN_STATE     =4'b0111;

//Internal regs/wires
reg [3:0]  r_state,             r_next_state;
reg [6:0]  r_return_change,      r_next_return_change;
reg [6:0]  r_product_price,      r_next_product_price;

//Reset+Update State
always@(posedge i_clk  or posedge i_rst)
   begin
   if(i_rst)
    begin
     r_state              <=IDLE_STATE;
     r_return_change       <=0;
     r_product_price       <=0;
    end
   else
    begin
     r_state          <=  r_next_state;
     r_return_change   <=  r_next_return_change;
     r_product_price   <=  r_next_product_price;
```

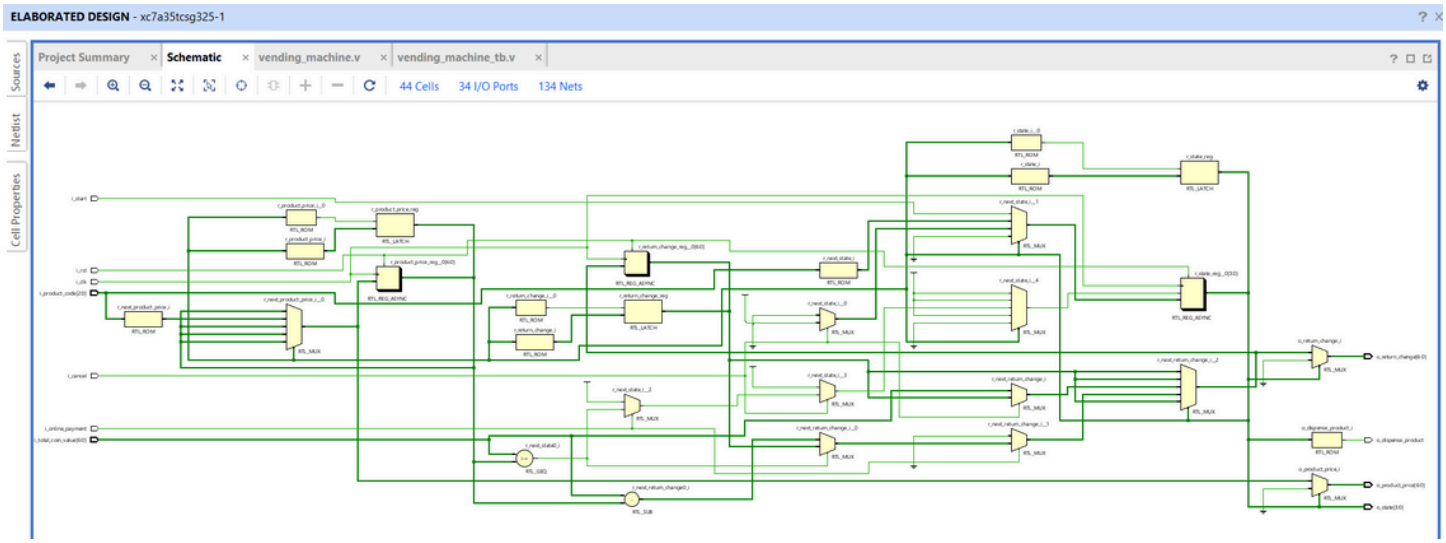
```
    end
end
//FSM
  always@(*)
   begin
  //Memory
     r_next_state        = r_state        ;
     r_next_return_change = r_return_change ;
     r_next_product_price = r_product_price ;

  case(r_state)

      IDLE_STATE: begin

        if(i_start)
          r_next_state = SELECT_PRODUCT_STATE;
        else if(i_cancel)
          r_next_state = IDLE_STATE;
        else
          r_next_state = IDLE_STATE;

       end

      SELECT_PRODUCT_STATE:begin

        case(i_product_code)
```

```
3'b000: begin
  r_next_state        =  KITKAT_SELECTION_STATE;
  r_next_product_price  =  KITKAT_PRICE;
end

3'b001: begin
  r_next_state        =  SNICKERS_SELECTION_STATE;
  r_next_product_price  =  SNICKERS_PRICE;
end

3'b010: begin
  r_next_state        =  FIVESTAR_SELECTION_STATE;
  r_next_product_price  =  FIVESTAR_PRICE;
end

3'b011: begin
  r_next_state        =  DIARYMILK_SELECTION_STATE;
  r_next_product_price  =  DIARYMILK_PRICE;
end

3'b100: begin
  r_next_state        =  FUSE_SELECTION_STATE;
  r_next_product_price  =  FUSE_PRICE;
end
```

```
      //Never reached
      default: begin
        r_next_state        =  IDLE_STATE;
        r_next_product_price  =  0;
      end
  endcase


end


   KITKAT_SELECTION_STATE,
   SNICKERS_SELECTION_STATE,
   FIVESTAR_SELECTION_STATE,
   DIARYMILK_SELECTION_STATE,
   FUSE_SELECTION_STATE     :
    begin


          if(i_cancel) begin
                    r_next_state = IDLE_STATE;
                    r_next_return_change = i_total_coin_value;
                  end
          else if(i_online_payment)
             r_next_state = DISPENSE_AND_RETURN_STATE;
          else if(i_total_coin_value >= r_product_price)
             r_next_state = DISPENSE_AND_RETURN_STATE;
          else
             r_next_state = r_state;
```

```
        end

      DISPENSE_AND_RETURN_STATE: begin

       r_next_state = IDLE_STATE;
       if(i_online_payment) begin
          r_next_return_change = 0;
          end
       else if(i_total_coin_value >= r_product_price)
          r_next_return_change = i_total_coin_value - r_product_price;
      end

        default:    begin

          r_state             = IDLE_STATE;
          r_return_change      = 0;
          r_product_price      = 0;
         end

  endcase

end

// Output Logic

assign o_state = r_state;
```

```
assign o_dispense_product =  (r_state == DISPENSE_AND_RETURN_STATE) ? 1'b1 : 1'b0;

assign o_product_price    =  (r_state == DISPENSE_AND_RETURN_STATE) ? r_next_product_price
: 1'b0;

assign o_return_change    =  (r_state == DISPENSE_AND_RETURN_STATE) ? r_next_return_change
: 1'b0;



endmodule
```



.

5. Test Benches and Their Results

5.1 Test Bench

`timescale 1ns / 1ps

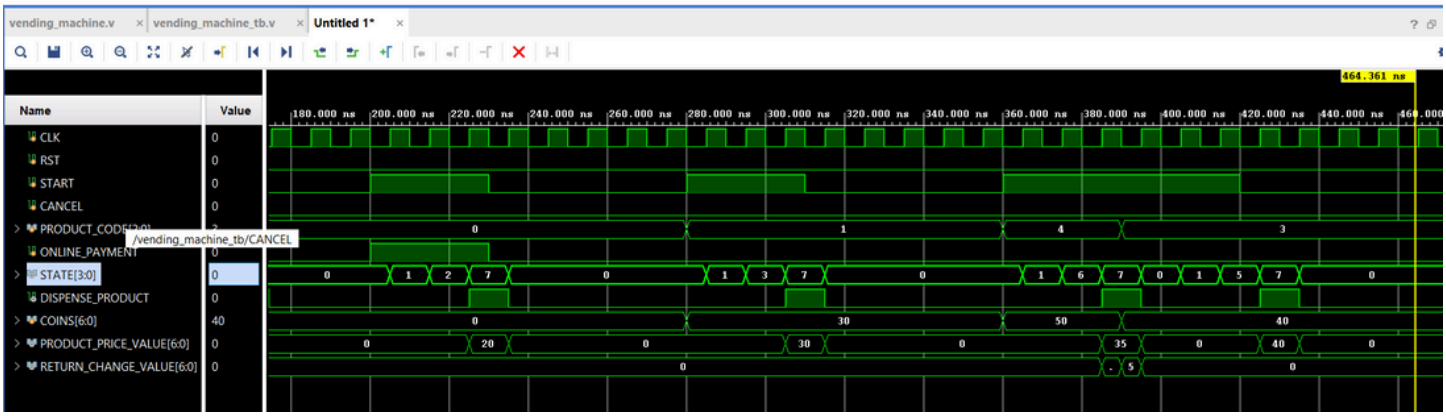
```
module vending_machine_tb();


//INPUT

reg CLK;

reg RST;

reg START;
```

```
reg CANCEL;
reg [2:0] PRODUCT_CODE;
reg [6:0] COINS;
reg ONLINE_PAYMENT;

//OUTPUT
wire [3:0] STATE;
wire DISPENSE_PRODUCT;
wire [6:0] PRODUCT_PRICE_VALUE;
wire [6:0] RETURN_CHANGE_VALUE;

//CLOCK SIGNAL
always #5 CLK = ~CLK;

initial begin
    CLK             =1'b0 ;
    RST             =1'b0 ;
    START           =1'b0 ;
    CANCEL          =1'b0 ;
    PRODUCT_CODE    =3'b0 ;
    COINS           =1'b0 ;
    ONLINE_PAYMENT  =1'b0 ;

  #100 RST = 1'b0;
  #100;
```

```
//KITKAT DISPENSE WITH ONLINE PAYMENT

START         = 1'b1;

ONLINE_PAYMENT = 1'b1;  //KITKAT

#30 START      = 1'b0;

ONLINE_PAYMENT = 1'b0;

#50;


START         = 1'b1;

PRODUCT_CODE   = 3'b001;    //SNICKERS

COINS         = 7'd30;

#30 START      = 1'b0;

#50;



START         = 1'b1;

PRODUCT_CODE   = 3'b100;    //FUSE

COINS         = 7'd50;

#30 START      = 1'b0;





START         = 1'b1;

PRODUCT_CODE   = 3'b011;    //DIARYMILK

COINS         = 7'd40;

#30 START      = 1'b0;
```

```
    #50;

    #50 $finish;

end

//INSTANTATION
vending_machine DUT  (

.i_clk              (CLK),                  //Clock signal (100MHz)
.i_rst              (RST),                  //Reset signal (Active High)
.i_start            (START),                //Initialize System
.i_cancel           (CANCEL),               //Cancel signal
.i_product_code     (PRODUCT_CODE),         //Product selection
.i_total_coin_value (COINS),                //Total value of the coin
.i_online_payment   (ONLINE_PAYMENT),       //Conformation of the online payment
.o_state            (STATE),                //Present state
.o_dispense_product (DISPENSE_PRODUCT),     //whether product dispensed
.o_product_price    (PRODUCT_PRICE_VALUE),  //product price
.o_return_change    (RETURN_CHANGE_VALUE)   //returned change
    );



endmodule
```

5.2 Simulation Results



6. Conclusion

This project successfully demonstrates the design and simulation of a vending machine controller using Verilog HDL. The FSM-based controller handles multiple products, various payment methods, change calculation, and cancellation. Simulation results confirm correct operation for all scenarios. The parameterized and modular approach allows for easy expansion to more products or features, such as additional payment methods or a user display. This work highlights the effectiveness of FSMs in digital control systems and provides a solid foundation for further enhancements or hardware implementation.